

ResearchBot

A Minor Project Report

Submitted in partial fulfillment of requirement of the

Degree of

**BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE &
ENGINEERING**

BY

Mahak Rathod: EN23CS301577

Mahak Sachdev: EN23CS301578

Manvi Agrawal: EN23CS301596

Under the Guidance of

Prof. Diptanshu Pandya



Department of Computer Science & Engineering

Faculty of Engineering

MEDICAPS UNIVERSITY, INDORE- 453331

APRIL-2026

ResearchBot

A Minor Project Report

Submitted in partial fulfillment of requirement of the

Degree of

**BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE &
ENGINEERING**

BY

Mahak Rathod: EN23CS301577

Mahak Sachdev: EN23CS301578

Manvi Agrawal: EN23CS301596

Under the Guidance of

Prof. Diptanshu Pandya



**Department of Computer Science & Engineering
Faculty of Engineering
MEDICAPS UNIVERSITY, INDORE- 453331**

APRIL-2026

Report Approval

The project work “**Rag ChatBot for Research paper**” is hereby approved as a creditable study of an engineering/computer application subject carried out and presented in a manner satisfactory to warrant its acceptance as prerequisite for the Degree for which it has been submitted.

It is to be understood that by this approval the undersigned do not endorse or approve any statement made, opinion expressed, or conclusion drawn there in; but approve the “Project Report” only for the purpose for which it has been submitted.

Internal Examiner

Name:

Designation

Affiliation

External Examiner

Name:

Designation

Affiliation

Declaration

I/We hereby declare that the project entitled **“Rag ChatBot for Research paper”** submitted in partial fulfillment for the award of the degree of Bachelor of Technology in ‘Computer Science and Engineering’ completed under the supervision of **Prof. Diptanshu Pandya, Assistant Professor, Department Computer Science and Engineering**, Faculty of Engineering, Medi-Caps University Indore is an authentic work.

Further, I/we declare that the content of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for the award of any degree or diploma.

Signature and name of the student(s) with date

Certificate

I/We, **Prof. Diptanshu Pandya** certify that the project entitled “**Rag ChatBot for Research paper**” submitted in partial fulfillment for the award of the degree of Bachelor of Technology/Master of Computer Applications by **Mahak Rathod, Mahak Sachdev, And Manvi Agrawal** is the record carried out by them under my/our guidance and that the work has not formed the basis of award of any other degree elsewhere.

Prof. Diptanshu Pandya

Computer Science and Engineering

Medi-Caps University, Indore

Dr. Kailash Chandra Bandhu

Head of the Department

Computer Science & Engineering

Medi-Caps University, Indore

Acknowledgements

I would like to express my deepest gratitude to the Honorable Chancellor, **Shri R C Mittal**, who has provided me with every facility to successfully carry out this project, and my profound indebtedness to **Prof. (Dr.) D. K. Patnaik**, Vice Chancellor, Medi-Caps University, whose unfailing support and enthusiasm has always boosted up my morale. I also thank **Prof. (Dr.) Ratnesh Litoriya**, Associate Dean (Computing and Allied Branch), Faculty of Engineering, Medi-Caps University, for giving me a chance to work on this project. I would also like to thank my Head of the Department **Dr. Kailash Chandra Bandhu** for his continuous encouragement for the betterment of the project.

Mahak Rathod,
Mahak Sachdev,
Manvi Agrawal

B.Tech. III Year (A)
Department of Computer Science & Engineering
Faculty of Engineering
Medi-Caps University, Indore

Abstract

In the modern era of artificial intelligence, automated document comprehension and intelligent question-answering has become increasingly important for researchers, students, and academic professionals. This mini project, ResearchBot, is a full-stack AI-powered web application that enables users to upload research papers in PDF format and interact with them through a natural language conversational interface using Retrieval-Augmented Generation (RAG) techniques.

Built on a client-server architecture, the system integrates a Streamlit-based frontend with a FastAPI backend, and leverages a local Ollama LLM (Phi model) combined with HuggingFace sentence-transformer embeddings and a ChromaDB vector store for intelligent document retrieval and answer generation. The pipeline incorporates ML-powered query enhancement, cross-encoder re-ranking, and cosine similarity-based evaluation.

ResearchBot supports two operating modes: Student/Faculty Mode for interactive Q&A with simplified explanations, key point extraction, and document summarization; and Researcher/Author Mode for advanced analysis including quality scoring, plagiarism detection, research trend analysis, and improvement suggestions. The system implements JWT-based authentication, persistent chat history, multi-document management, and a structured ML evaluation pipeline with accuracy, precision, recall, and F1 metrics.

Keywords

Keyword	Explanation
RAG (Retrieval-Augmented Generation)	A technique combining document retrieval with LLM generation for context-aware answers.
Ollama (Phi)	A locally-hosted open-source LLM used for text generation and reasoning.
ChromaDB	An in-memory vector database used to store and retrieve document embeddings.
HuggingFace Embeddings	Sentence-transformer models that convert text chunks into dense vector representations.
Cross-Encoder Re-ranking	A BERT-based model that re-scores retrieved documents for improved relevance.
FastAPI	A Python web framework for building high-performance REST APIs on the backend.
Streamlit	A Python library used to build the interactive multi-mode frontend UI.
LangChain	A framework that orchestrates LLM chains, prompts, retrievers, and document loaders.

Keyword	Explanation
PyPDF	A Python library used to parse and extract text from uploaded PDF research papers.
Cosine Similarity	A metric used to measure semantic closeness between two embedding vectors.

Table of Contents

		Page No.
	Front page	I
	Report Approval	II
	Declaration	III
	Certificate	IV
	Acknowledgement	V
	Abstract	VI
	Table of Contents	VIII
	List of figures	X
	Abbreviations	XI
Chapter 1	Introduction (Whichever is applicable)	
	1.1 Introduction	
	1.2 Literature Review	
	1.3 Objectives	
	1.4 Significance	
	1.5 Research Design	
	1.6 Source of Data	
	1.7 Chapter Scheme	
Chapter 2	REQUIREMENTS SPECIFICATION	
	2.1 User Characteris	
	2.2 Functional Requirements	
	2.3 Dependencies	
	2.4 Performance Requirements	

	2.5 Hardware Requirements	
	2.6 Constraints & Assumptions	
Chapter 3	DESIGN (Whichever is applicable)	
	3.1 Algorithm (if Applicable)	
	3.2 Function Oriented Design for procedural approach	
	3.3 System Design (Whichever is applicable)	
	3.3.1 Data Flow Diagrams (Level 0,Level1)	
	3.3.2 Activity Diagram	
	3.3.3 Flow Chart	
	3.3.4 Class Diagram	
	3.3.5 ER Diagram	
	3.3.6 Sequence diagram	
	3.4 Database Design	
	3.4.1 Logical Database Design	
	3.4.2 Physical Database Design	
Chapter 4	Implementation, Testing, and Maintenance	
	4.1 Introduction to Languages, IDE's, Tools and Technologies used for Implementation	
	4.2 Testing Techniques and Test Plans (According to project)	
	4.3 Installation Instructions	
	4.4 End User Instructions	
Chapter 5	Results and Discussions	
	5.1 User Interface Representation (of Respective Project)	
	5.2 Brief Description of Various Modules of the system	
	5.3 Snapshots of system with brief detail of each	

	5.4 Back Ends Representation (Database to be used)	
	5.5 Snapshots of Database Tables with brief description	
Chapter 6	Summary and Conclusions	
Chapter 7	Future scope	
	Appendix	

List of Figures

Figure No.	Name of the Figure
Fig 3.0	File Structure Diagram
Fig 3.1	Data Flow Diagram – Level 0
Fig 3.2	Data Flow Diagram – Level 1
Fig 3.3	Flow Chart Diagram
Fig 3.4	Sequence Diagram
Fig 3.5	Component Diagram
Fig 3.6	Use-Case Diagram
Fig 3.7	Class Diagram
Fig 3.8	Activity Diagram

Abbreviations

- AI – Artificial Intelligence
- RAG – Retrieval-Augmented Generation
- API – Application Programming Interface
- LLM – Large Language Model
- NLP – Natural Language Processing
- UI – User Interface
- DB – Database
- URI – Uniform Resource Identifier
- URL – Uniform Resource Locator
- HTTP – Hypertext Transfer Protocol
- JSON – JavaScript Object Notation
- CORS – Cross-Origin Resource Sharing
- PDF – Portable Document Format
- JWT – JSON Web Token
- QA – Question Answering
- T2T – Text-to-Text (generation)
- VDB – Vector Database
- DFD – Data Flow Diagram
- UML – Unified Modeling Language
- ER – Entity Relationship
- FAISS – Facebook AI Similarity Search
- ChromaDB – Chroma Vector Database
- HF – HuggingFace
- MiniLM – Mini Language Model (sentence-transformers) •
- CE – Cross-Encoder (re-ranking model)
- ENV – Environment Variable
- PORT – Port Number
- req – Request Object (FastAPI/Python)
- res – Response Object (FastAPI/Python)
- PyPDF – Python PDF Parsing Library
- EMU – English Metric Unit (document sizing) • ELI5 – Explain Like I am 5 (simplified explanation feature) • F1 – F1 Score (harmonic mean of precision and recall) • k – Number of top documents retrieved (retrieval_top_k = 3) • cosine – Cosine Similarity metric for embedding comparison

Chapter 1: Introduction

1.1 Introduction

Artificial Intelligence has significantly transformed academic research workflows, particularly through Retrieval-Augmented Generation (RAG) systems that combine document retrieval with large language model generation to produce contextually accurate, grounded answers from domain-specific documents. Unlike purely generative models that may hallucinate, RAG architectures anchor responses in retrieved source material, making them particularly well-suited for academic and research applications.

ResearchBot is a web-based AI-powered research paper analysis system designed to provide researchers, students, and faculty with an accessible, intelligent, and multi-functional platform for interacting with uploaded PDF research papers. Instead of training large generative models from scratch, the system integrates a locally-hosted Ollama Phi LLM, HuggingFace sentence-transformer embeddings, and a ChromaDB vector database to deliver high-quality document understanding with minimal infrastructure overhead.

The system follows a client-server architecture where the Streamlit frontend collects user prompts and manages document state, while the FastAPI backend orchestrates the full RAG pipeline: query enhancement, vector retrieval, cross-encoder re-ranking, LLM-based answer generation, and ML evaluation. ResearchBot supports two distinct user modes — Student/Faculty Mode for simplified Q&A and Researcher/Author Mode for advanced paper analysis — ensuring it serves a diverse academic user base.

1.2 Literature Review

Question answering over documents is a well-studied problem in natural language processing. Early approaches relied on keyword-based retrieval and rule-based extraction, which could not handle semantic variation in user queries. The advent of transformer-based models such as BERT enabled dense passage retrieval, significantly improving semantic matching between queries and document chunks.

The RAG paradigm, introduced by Lewis et al. (2020), combines a retrieval component with a generative language model, allowing systems to produce factual and contextually grounded answers from external document corpora. Subsequent work has demonstrated RAG's effectiveness in domain-specific question answering, particularly in scientific and academic settings.

Re-ranking using cross-encoder models has been shown to substantially improve retrieval precision by scoring query-document pairs jointly rather than independently. Models such as ms-marco-MiniLM cross-encoders provide competitive re-ranking performance at low computational cost, making them suitable for interactive systems like ResearchBot.

Local inference using lightweight open-source LLMs such as Phi (Microsoft) via Ollama has emerged as a privacy-preserving and cost-effective alternative to cloud-based API services. Combined with efficient vector stores like ChromaDB and sentence-transformers for embedding, such architectures enable capable academic AI assistants deployable on standard hardware.

1.3 Objectives

Main Objective:

To design and develop a multi-mode AI-powered research paper analysis system, ResearchBot, that enables users to upload PDF documents, ask natural language questions, and receive contextually grounded answers through a complete Retrieval-Augmented Generation pipeline with ML transparency.

Supporting Objectives:

- To analyze user requirements for intelligent academic document Q&A across student and researcher personas.
- To design a modular client-server architecture integrating a local LLM, vector database, and re-ranking pipeline.
- To implement secure JWT-based authentication and persistent multi-document management.
- To develop a Student Mode with simplified explanations, key point extraction, and document summarization.
- To develop a Researcher Mode with quality scoring, plagiarism detection, trend analysis, and improvement suggestions.
- To implement a structured ML evaluation pipeline measuring accuracy, precision, recall, and F1 score.
- To provide a responsive, multi-mode user interface through Streamlit.

1.4 Significance

ResearchBot addresses the growing need for intelligent, document-aware AI assistants among students, researchers, and academic professionals who require deep interaction with research literature. Traditional PDF readers and search tools offer no semantic understanding — ResearchBot bridges this gap by embedding AI-powered comprehension directly into the document interaction workflow.

The dual-mode design ensures the platform serves both entry-level users who need simplified explanations and advanced researchers who require analytical tools such as plagiarism detection and trend analysis. The local LLM approach ensures data privacy, while the modular pipeline design makes the system extensible and maintainable.

1.5 Research Design

The project follows an applied research and development approach, focused on implementing a practical RAG solution using existing pre-trained models and frameworks. The research design includes:

- Studying state-of-the-art RAG architectures and cross-encoder re-ranking methods.
- Designing a modular client-server architecture using Streamlit and FastAPI.
- Implementing a multi-step ML pipeline: query enhancement → retrieval → re-ranking → generation.
- Integrating a local Ollama LLM for private, on-device inference.
- Developing a structured evaluation framework with standard academic QA metrics.
- Testing performance, usability, and accuracy of the complete system.

1.6 Source of Data

- User-uploaded PDF research papers (primary document corpus).

- HuggingFace pre-trained sentence-transformer embeddings.
- Standard ML evaluation Q&A test set (10 predefined academic questions).
- Research literature on RAG, cross-encoder re-ranking, and LLM-based QA systems.

1.7 Chapter Scheme

Chapter 2 presents the Requirement Specification.

Chapter 3 discusses the System Design and architecture diagrams.

Chapter 4 covers Implementation, Testing, and Maintenance.

Chapter 5 presents Results and Discussion.

Chapter 6 provides Summary and Conclusions.

Chapter 7 outlines Future Scope.

Chapter 2: Requirement Specification

2.1 User Characteristics

The primary users of ResearchBot include:

- Students requiring intelligent Q&A assistance for understanding and summarizing research papers.
- Faculty members analyzing academic literature and preparing course materials.
- Researchers evaluating paper quality, detecting redundancy, and identifying improvement areas.
- Academic writers requiring trend analysis and structured improvement suggestions.
- Developers and AI enthusiasts exploring RAG pipelines and local LLM integration.

Users are expected to:

- Have basic knowledge of web applications and PDF document management.
- Be able to formulate natural language questions about research content.
- Understand basic authentication processes (registration/login).

2.2 Functional Requirements

1. User Management

- Register new users with username and password.
- Hash passwords securely before storage.
- Authenticate users via login using credential validation.
- Generate JWT tokens upon successful authentication.
- Protect sensitive routes using authentication middleware.

2. Document Management

- Accept PDF file uploads from authenticated users.
- Parse and extract text from PDFs using PyPDF.
- Split documents into overlapping chunks (500 chars, 100 overlap) using LangChain.
- Generate and store embeddings in ChromaDB vector database.
- Support switching between multiple uploaded documents.
- Display uploaded file history per user.

3. RAG Pipeline — Student/Faculty Mode

- Accept natural language questions from authenticated users.
- Enhance query using LLM-based query rewriting (Step 1).
- Retrieve top-k (k=3) relevant chunks from ChromaDB (Step 2).
- Re-rank retrieved documents using cross-encoder ms-marco-MiniLM (Step 3).
- Generate answers using Ollama Phi LLM from top-2 re-ranked chunks (Step 4).
- Return simplified explanation, document summary, and key points in Student Mode.
- Display full ML pipeline details (original → enhanced → retrieved → reranked → answer).

4. Research Analyzer — Researcher/Author Mode

- POST /analyze/quality — LLM-based quality scoring (clarity, novelty, readability, overall).
- POST /analyze/plagiarism — Embedding cosine similarity-based redundancy detection.
- POST /analyze/trends — LLM-based trend relevance and gap analysis.
- POST /analyze/suggestions — LLM-generated section-wise improvement recommendations.

5. ML Evaluation

- POST /evaluate — Asynchronous evaluation pipeline using 10 standard academic questions.
- Compare RAG model answers against LLM-generated reference answers via cosine similarity.
- Compute accuracy, precision, recall, F1, average similarity, and cache hit rate.
- Support quick (3 questions) and full (10 questions) evaluation modes.
- Display Plotly-based visualizations: similarity distribution and accuracy pie chart.

6. API Structure

- POST /upload — Document upload and vector store creation.
- POST /ask — RAG question answering endpoint.
- POST /login, POST /signup — Authentication endpoints.
- GET /history — Retrieve user chat history.
- GET /files — List uploaded files per user.
- POST /set_file — Switch active document.
- POST /evaluate — Start background evaluation job.
- GET /status/{job_id} — Poll evaluation progress.
- GET /dashboard — Retrieve evaluation state.

2.3 Dependencies

- Ollama runtime installed locally with the Phi model pulled.
- Internet connectivity for initial HuggingFace model downloads.
- Python 3.9+ runtime environment.
- FastAPI and Uvicorn for backend server.
- Streamlit for frontend UI.
- LangChain, LangChain-Community, LangChain-HuggingFace, LangChain-Ollama.
- ChromaDB for in-memory vector storage.
- Sentence-Transformers for embedding and cross-encoder models.
- PyPDF for PDF parsing.
- scikit-learn and NumPy for evaluation metric computation.
- Plotly for evaluation visualization in the frontend.

2.4 Performance Requirements

- Authentication requests must be processed within minimal latency.

- PDF parsing and embedding generation should complete within acceptable time for files up to 50 pages.
- RAG query answering should respond asynchronously with loading indicators.
- Evaluation pipeline must run as a background job with real-time progress reporting.
- Embedding similarity computations must be optimized using NumPy matrix operations.
- The system must support multiple concurrent users with isolated document contexts.

2.5 Hardware Requirements

Server Side

- Processor: Intel i5 / AMD Ryzen 5 or higher (i7 recommended for Ollama inference).
- RAM: Minimum 8 GB (16 GB recommended for local LLM and embedding models).
- Storage: Minimum 5 GB (for Ollama Phi model + embeddings).
- Operating System: Windows / Linux / macOS.

Client Side

- Modern web browser (Chrome, Edge, Firefox, Safari).
- Stable internet connection for initial model downloads.

2.6 Constraints & Assumptions

Constraints

- System requires Ollama runtime and Phi model to be installed and running locally.
- ChromaDB operates in-memory; embeddings are not persisted across server restarts.
- LLM response quality depends on the capabilities of the locally hosted Phi model.
- System scalability is influenced by local hardware constraints for inference.
- PDF parsing quality depends on the structure and formatting of the uploaded file.

Assumptions

- Users upload well-structured PDF research papers in English.
- The Ollama Phi model is available and responsive on port 11434.
- Authentication tokens are securely stored in the client session state.
- Users comply with academic integrity and usage guidelines.

Chapter 3: System Design

3.1 Algorithm (RAG Pipeline Flow)

1. User Registration / Login with JWT authentication.
2. User uploads PDF research paper.
3. Backend parses PDF using PyPDF and splits into overlapping 500-character chunks.
4. HuggingFace MiniLM embeddings generated for each chunk and stored in ChromaDB.
5. User submits a natural language question in Student or Researcher mode.
6. Backend validates JWT and forwards question to RAG pipeline.
7. Step 1 — Query Enhancement: Ollama Phi LLM rewrites query for retrieval clarity.
8. Step 2 — Vector Retrieval: ChromaDB retrieves top-3 semantically similar chunks.
9. Step 3 — Cross-Encoder Re-ranking: ms-marco-MiniLM re-scores and re-ranks retrieved documents.
10. Step 4 — Answer Generation: Ollama Phi LLM generates answer from top-2 re-ranked chunks.
11. Student Mode: Additional summary, key points, and simplified explanation generated.
12. Researcher Mode: Quality scores, plagiarism check, trend analysis, and suggestions generated.
13. ML Evaluation: 10 standard questions evaluated via cosine similarity; metrics computed and visualized.

3.2 Function-Oriented Design (Modules)

- User Management Module — Registration, login, JWT authentication, session persistence.
- Document Management Module — PDF upload, PyPDF parsing, chunk splitting, ChromaDB ingestion.
- Query Enhancement Module — Ollama-based query rewriting for improved retrieval accuracy.
- Retrieval Module — ChromaDB vector similarity search (top-k=3 retrieval).
- Re-ranking Module — Cross-encoder scoring and re-ordering of retrieved documents.
- Answer Generation Module — LLM-based context-grounded answer generation.
- Student Mode Module — Summary generation, key point extraction, simplified explanation.
- Research Analyzer Module — Quality index, plagiarism detection, trend analysis, suggestions.
- Evaluation Module — Standard QA evaluation with accuracy, precision, recall, F1 metrics.
- Frontend Module — Streamlit multi-mode UI, prompt input, ML pipeline display, visualizations.

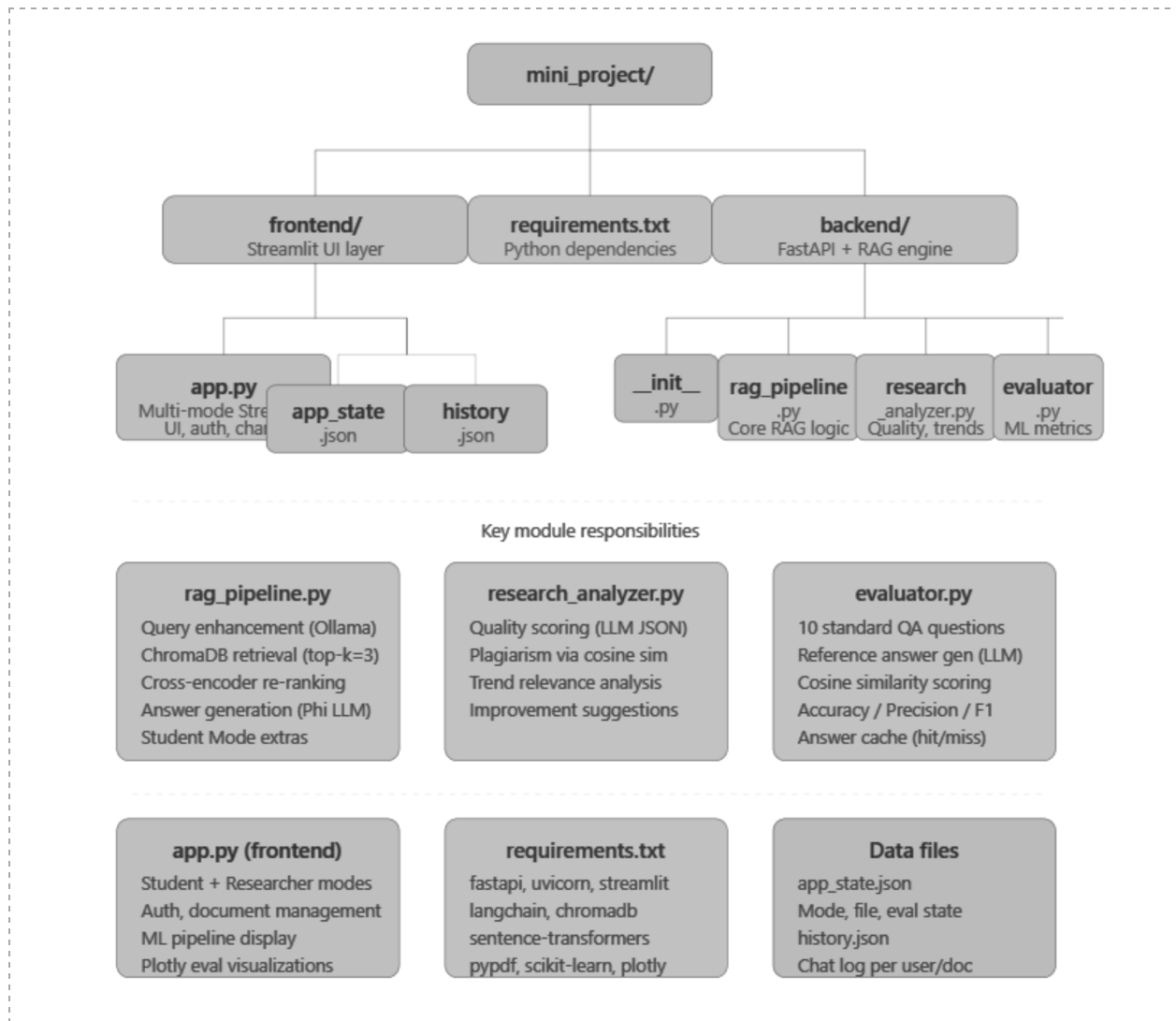


Fig 3.0 File Structure Diagram

3.3 System Architecture

Architecture Type: Client–Server with Local LLM Integration

- Frontend: Streamlit (multi-mode UI, prompt input, document management, ML pipeline display).
- Backend: FastAPI (REST APIs, RAG orchestration, authentication, evaluation jobs).
- Vector Store: ChromaDB in-memory (document chunk embeddings, semantic retrieval).
- Embedding Model: HuggingFace sentence-transformers/all-MiniLM-L6-v2.
- Re-ranking Model: cross-encoder/ms-marco-MiniLM-L-6-v2.
- LLM: Ollama Phi (local inference on port 11434).
- PDF Parsing: PyPDF via LangChain PyPDFLoader.

3.4 Data Flow Design

Level 0 DFD:

User → Streamlit Frontend → FastAPI Backend → RAG Pipeline (ChromaDB + Ollama) →
FastAPI Backend → Streamlit Frontend → User

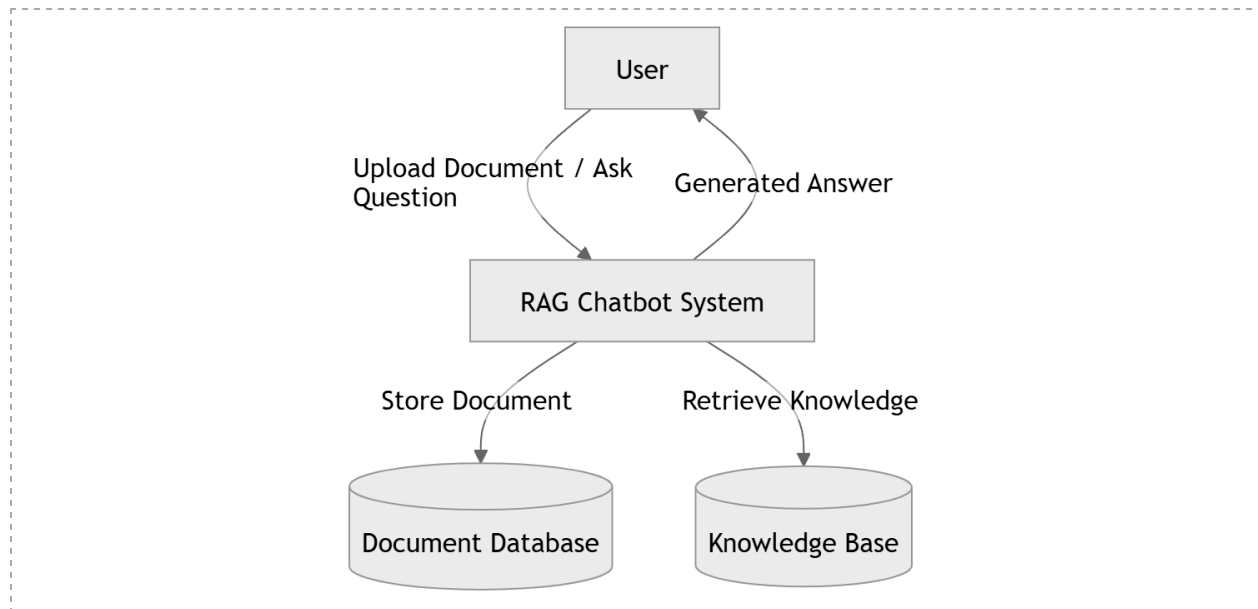


Fig 3.1 Data Flow Diagram – Level 0

Level 1 DFD:

- Validate JWT Token
- Parse and Chunk PDF Document
- Generate and Store Embeddings in ChromaDB
- Enhance Query using Ollama LLM
- Retrieve Top-k Chunks from ChromaDB
- Re-rank Documents using Cross-Encoder
- Generate Answer using Ollama Phi
- Return Structured Response to Frontend

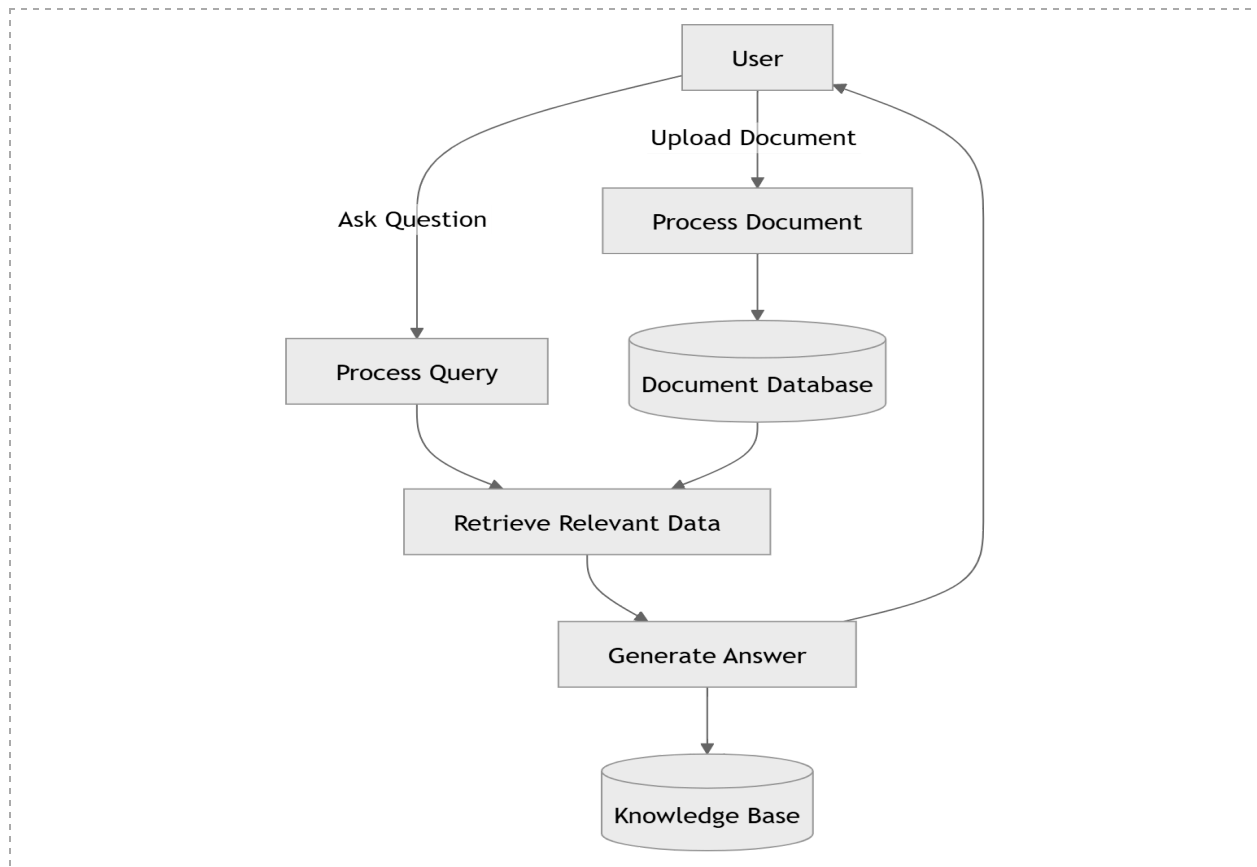


Fig 3.2 Data Flow Diagram – Level 1

3.5 UML Design Components

Use Cases:

- Register / Login
- Upload PDF Research Paper
- Switch Active Document
- Ask Question (Student Mode)
- View ML Pipeline Details
- Analyze Paper Quality
- Check Plagiarism
- Analyze Research Trends
- Get Improvement Suggestions
- Run ML Evaluation

Main Classes:

- User — username, password_hash, uploaded_files, chat_history.
- Document — file_path, chunks, embeddings, chroma_db.
- RAGPipeline — enhance_query(), retrieve(), rerank(), generate_answer().
- ResearchAnalyzer — analyze_quality(), detect_plagiarism(), analyze_trends(), generate_suggestions().
- Evaluator — run_full_evaluation(), compute_metrics(), get_dataset_info().

- AppState — mode, loaded_file, username, eval_result.

Sequence Flow:

User → Streamlit Frontend → FastAPI Backend → JWT Middleware → ChromaDB → Ollama LLM → FastAPI Backend → Streamlit Frontend

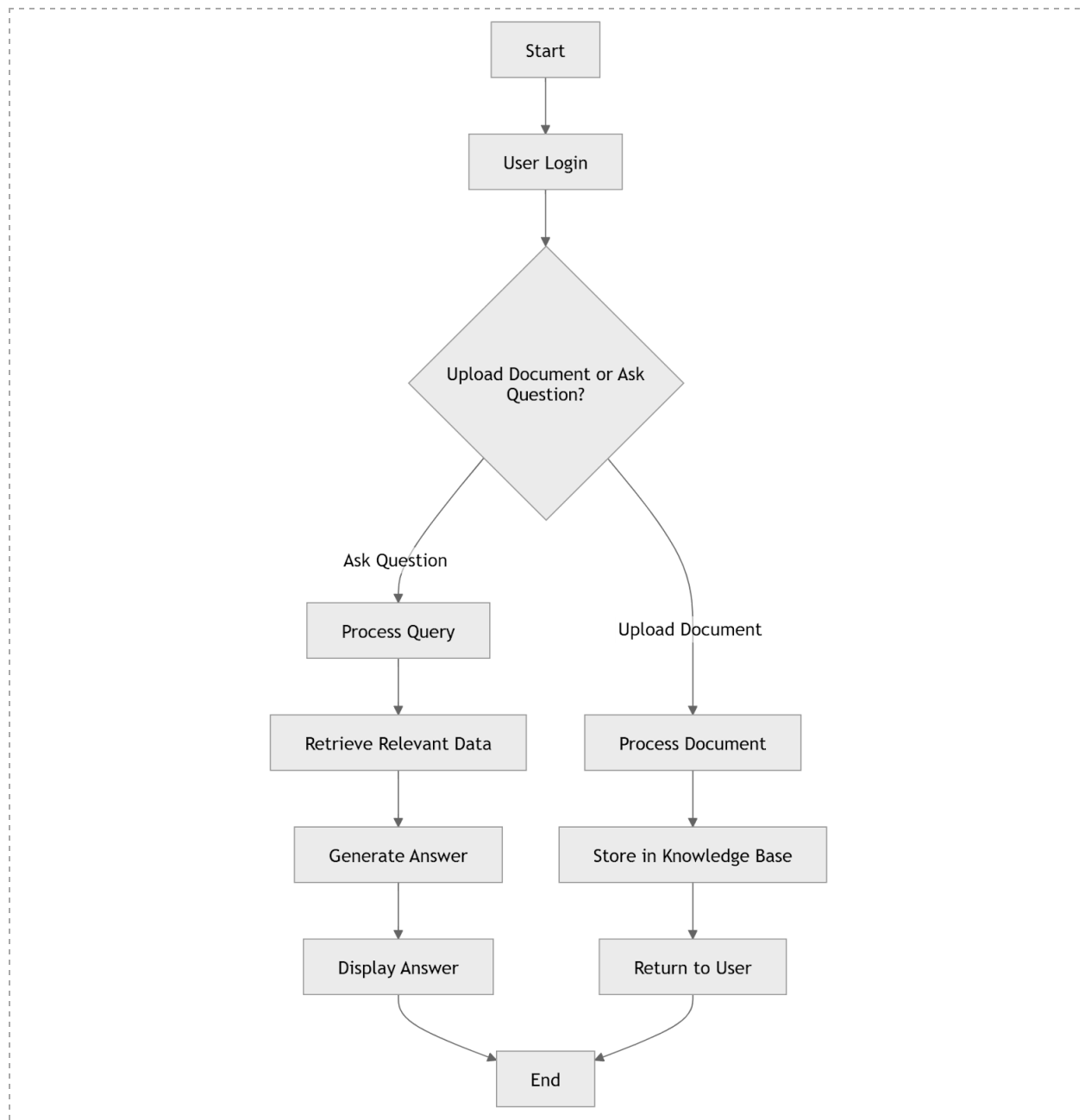


Fig 3.3 Flow Chart Diagram

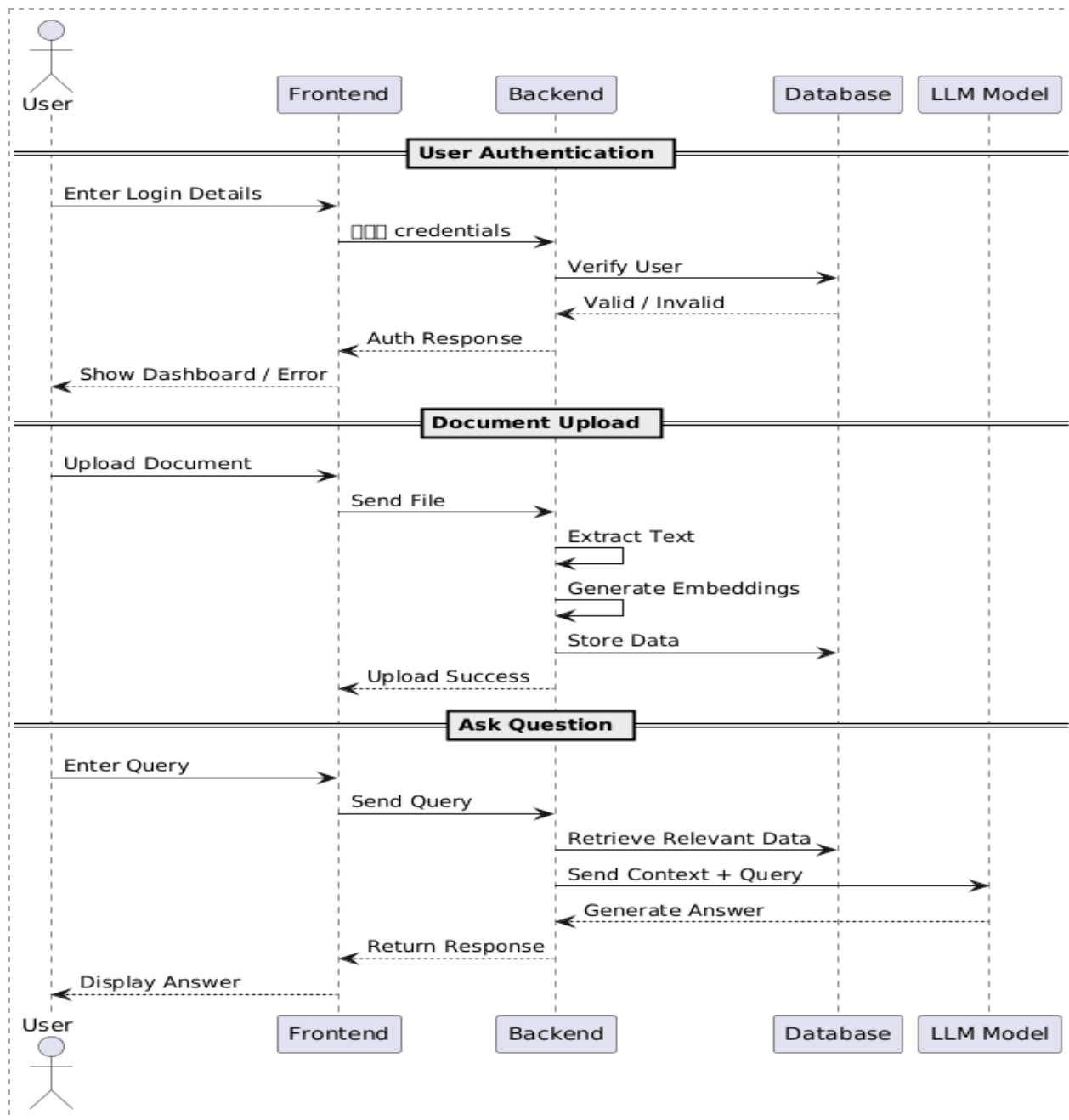


Fig 3.4 Sequence Diagram

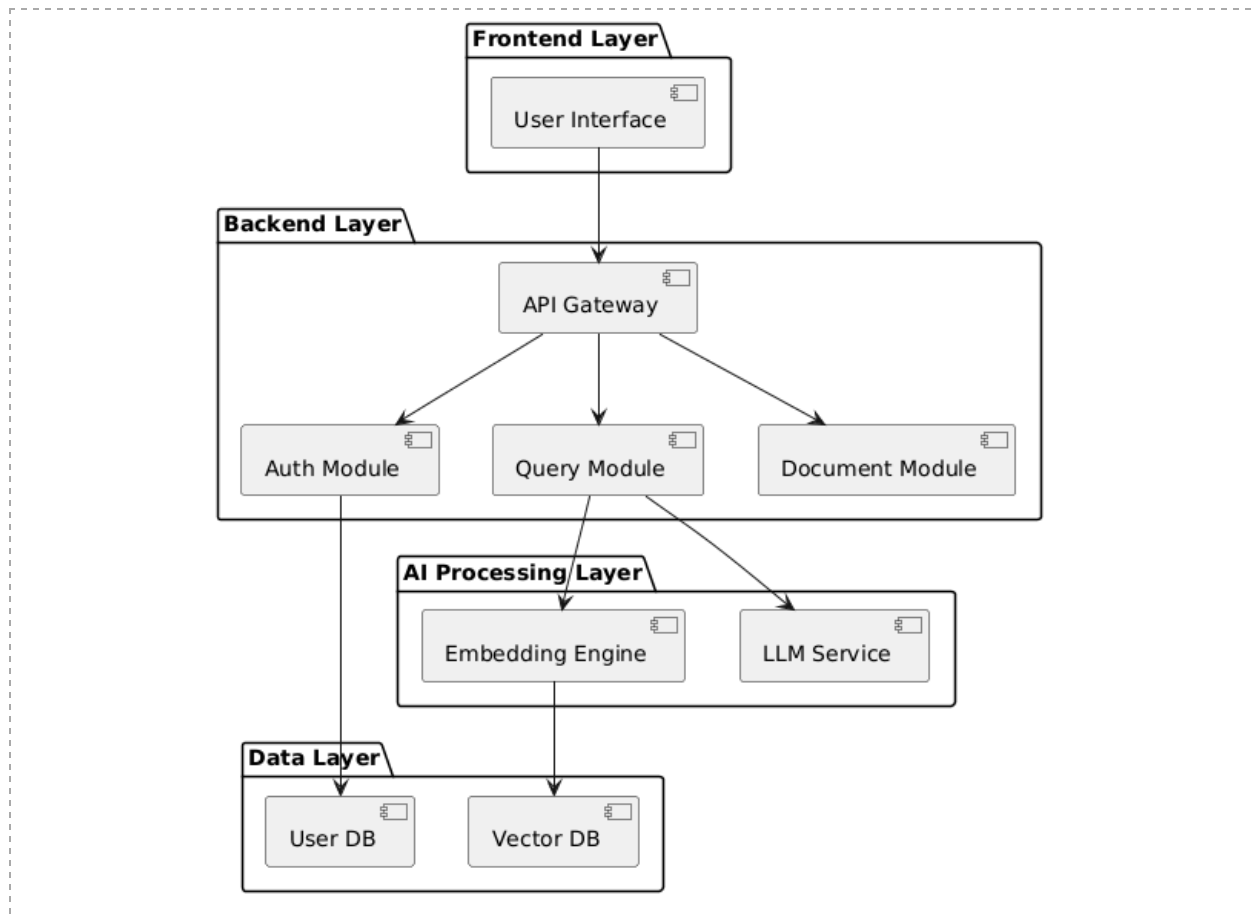


Fig 3.5 Component Diagram

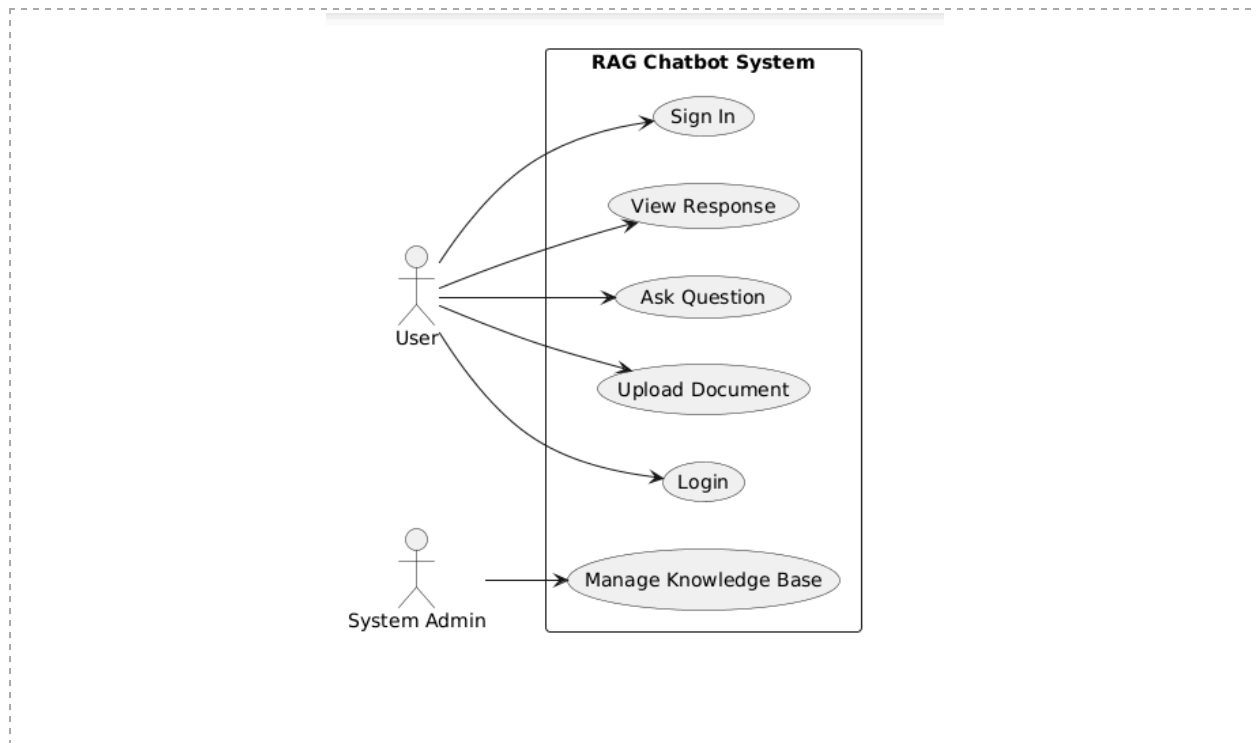


Fig 3.6 Use-Case Diagram

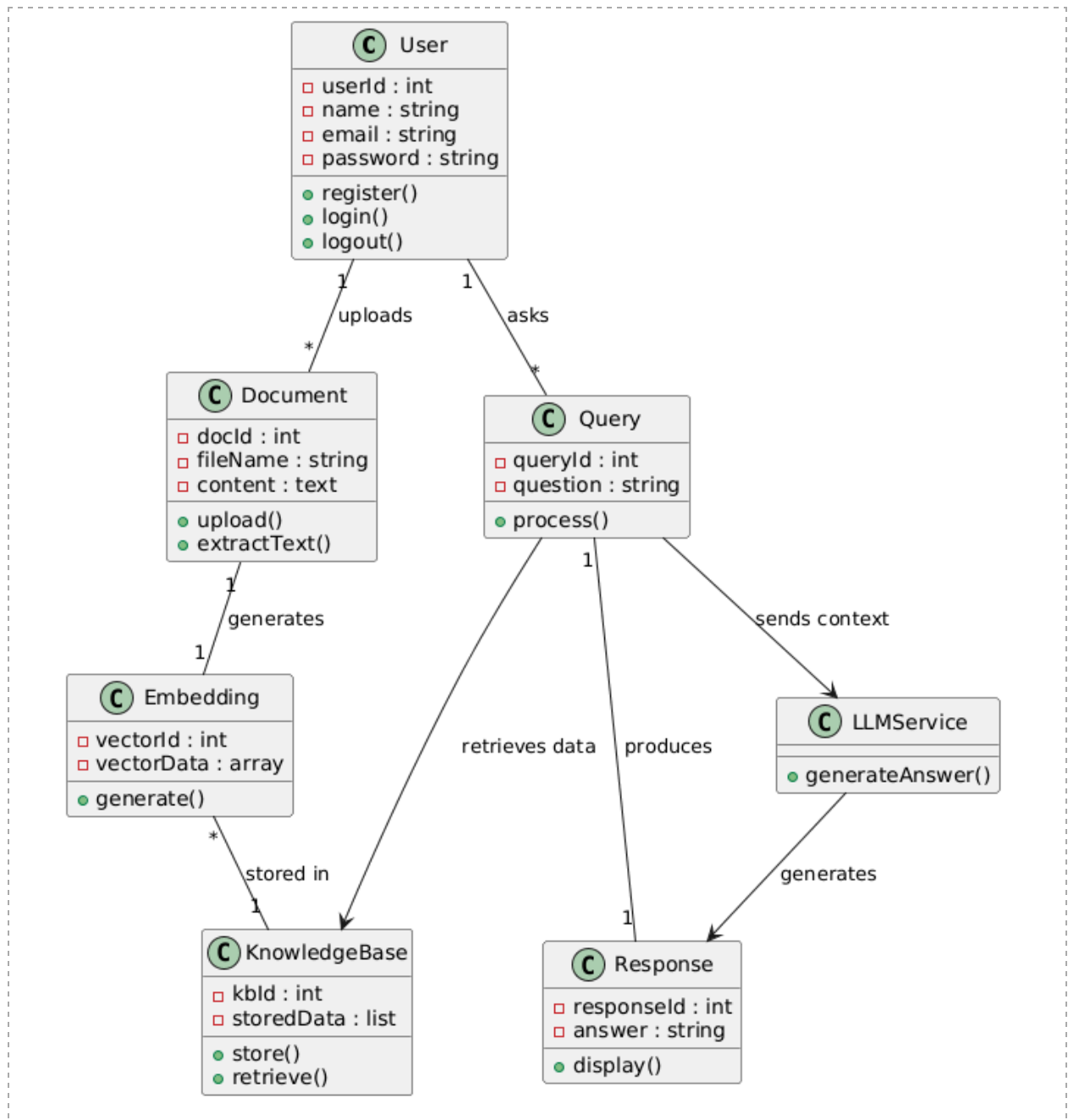


Fig 3.7 Class Diagram

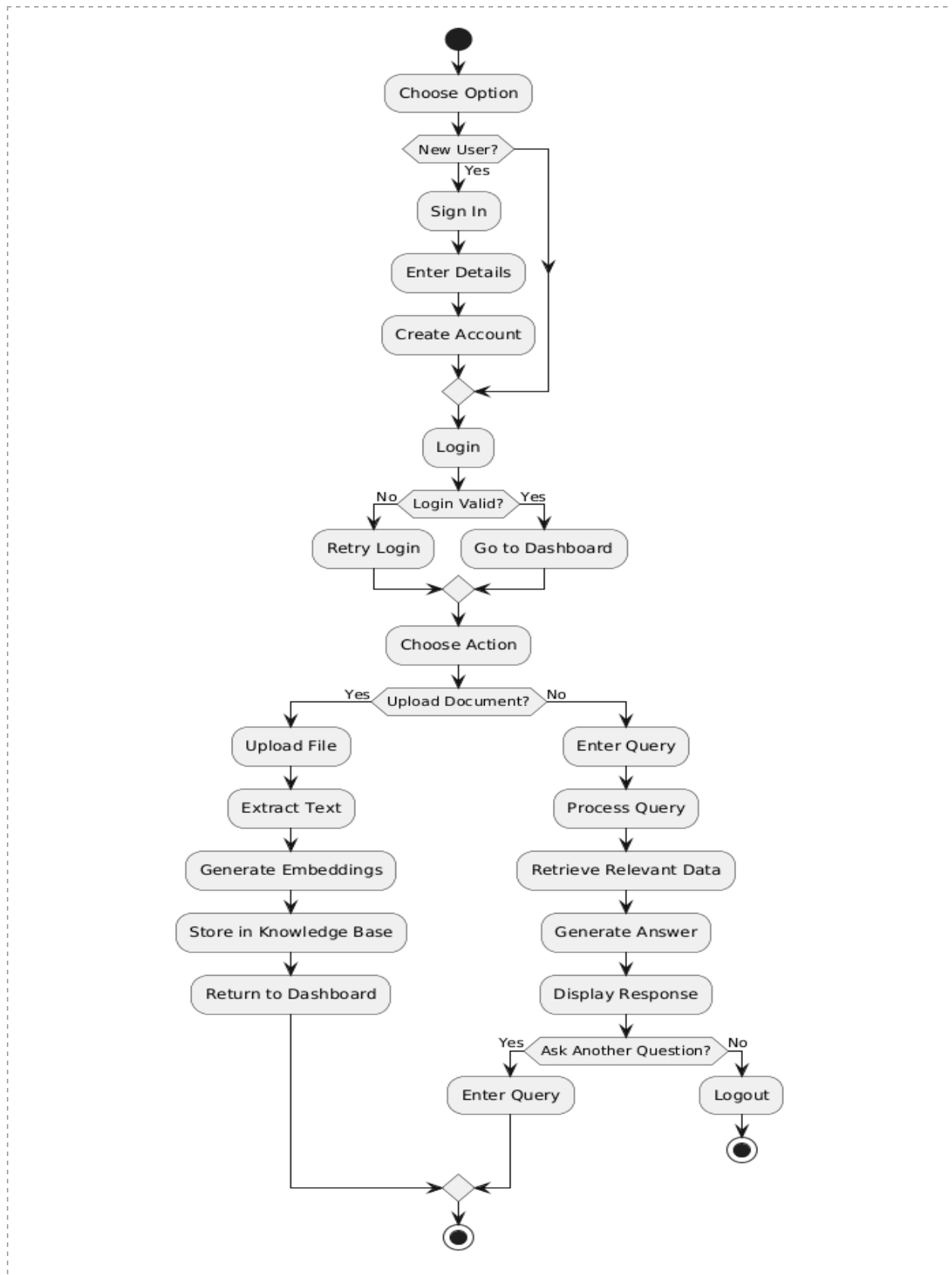


Fig 3.8 Activity Diagram

3.6 Non-Functional Design Considerations

- Security: JWT authentication, bcrypt password hashing, secure session state.
- Performance: Asynchronous evaluation jobs, cached embedding models, NumPy vectorized similarity.
- Scalability: Stateless JWT auth, modular pipeline, extensible vector store.
- Reliability: Structured error handling, fallback responses for LLM failures, graceful degradation.
- Privacy: Local Ollama LLM inference — no data sent to external cloud APIs.

3.7 Database Design

3.7.1 Logical Database Design

ResearchBot uses an in-memory ChromaDB vector store for document embeddings and a JSON-based file store for user data and application state.

Entity	Attributes	Description
User	username, password_hash, uploaded_files[]	Stores user credentials and uploaded file list
Document Chunk	content, embedding[], source, page	Vector store entry for each PDF chunk
Chat History	username, filename, role, content, timestamp	Persistent conversation log per user/document
App State	mode, loaded_file, username, eval_result	Session state persisted to app_state.json
Evaluation Result	metrics{}, results[], dataset_info{}, model_info{}	ML evaluation output cache

3.7.2 Physical Database Design

- ChromaDB: In-memory vector store instantiated per uploaded document session.
- history.json: Flat JSON file storing chat history entries per user and filename.
- app_state.json: Flat JSON file storing current application mode and loaded file state.
- User data: Stored in memory during server session (future: persistent DB recommended).

Chapter 4: Implementation, Testing, and Maintenance

4.1 Languages, IDEs, Tools, and Technologies Used

ResearchBot is implemented using a Python-first full-stack architecture following a client-server model. The system integrates frontend technologies, backend frameworks, vector database management, local LLM inference, and ML evaluation components.

Frontend & Backend Technologies

- Frontend: Streamlit (multi-mode UI), Plotly (evaluation visualizations).
- Backend: FastAPI with Uvicorn ASGI server.
- Vector Database: ChromaDB (in-memory).
- LLM: Ollama Phi (local inference, no cloud API required).
- Embedding Model: sentence-transformers/all-MiniLM-L6-v2 (HuggingFace).
- Re-ranking Model: cross-encoder/ms-marco-MiniLM-L-6-v2.
- RAG Orchestration: LangChain (loaders, splitters, vector stores, prompts, retrievers).
- PDF Parsing: PyPDF via LangChain PyPDFLoader.

Programming Languages

- Python 3.9+ — used for both frontend (Streamlit) and backend (FastAPI).
- JSON — used for chat history, app state, and evaluation result persistence.

Key Libraries

Library	Version	Purpose
fastapi	Latest	Backend REST API framework
uvicorn	Latest	ASGI server for FastAPI
streamlit	Latest	Frontend multi-mode UI
langchain	Latest	RAG orchestration framework
langchain-ollama	Latest	Ollama LLM integration
langchain-huggingface	Latest	HuggingFace embeddings
langchain-community	Latest	PDF loaders and vector stores
chromadb	Latest	In-memory vector database
sentence-transformers	Latest	Embeddings and cross-encoder
pypdf	Latest	PDF text extraction
scikit-learn	Latest	Cosine similarity and ML metrics
numpy	Latest	Numerical array operations
plotly	Latest	Evaluation visualization charts
pydantic	Latest	Request/response data validation

Library	Version	Purpose
requests	Latest	Frontend-to-backend HTTP client

Development Tools

- VS Code — Integrated Development Environment.
- Ollama — Local LLM runtime (phi model).
- GitHub — Version control and repository management.
- Postman — API testing during development.
- Python pip — Package management.

4.2 Testing Techniques and Test Plans

Unit Testing

- PDF parsing and chunk splitting logic tested with sample research papers.
- Embedding generation and ChromaDB insertion verified.
- Query enhancement prompt tested for correctness and fallback behavior.
- Cross-encoder re-ranking verified for correct score ordering.
- JWT token generation and validation middleware tested.
- Quality scoring JSON parsing and fallback defaults tested.
- Plagiarism cosine similarity threshold logic validated.
- Evaluation metric computation (accuracy, precision, recall, F1) verified.

Integration Testing

- Streamlit frontend to FastAPI backend communication tested using Requests.
- FastAPI to ChromaDB integration verified for chunk storage and retrieval.
- FastAPI to Ollama LLM integration verified for query enhancement and answer generation.
- End-to-end RAG pipeline flow tested: upload → embed → query → retrieve → rerank → generate → respond.
- Background evaluation job polling tested for correct progress reporting.

User Testing

- Real research paper PDFs uploaded and Q&A flow tested in Student Mode.
- Researcher Mode analysis tools tested with published academic papers.
- ML evaluation quick and full modes tested for correct metric output.
- UI responsiveness and loading indicators evaluated across modes.

Performance Testing

- Embedding model singleton pattern verified for minimal re-loading overhead.
- Evaluation answer cache tested for reduced LLM calls on repeated questions.
- Concurrent request handling tested for user isolation.
- External LLM failure handling validated for graceful fallback responses.

4.3 Installation Instructions

Backend Setup

14. Install Python 3.9 or higher.
15. Install Ollama from <https://ollama.ai> and run: `ollama pull phi`
16. Navigate to the backend directory.
17. Install required dependencies: `pip install -r requirements.txt`
18. Start the FastAPI backend server: `uvicorn main:app --reload --port 8000`

Frontend Setup

19. Navigate to the frontend directory.
20. Install required dependencies: `pip install streamlit plotly requests`
21. Start the Streamlit frontend: `streamlit run app.py`
22. Open the application in the browser (default: `http://localhost:8501`).
23. Ensure Ollama is running locally on port 11434 before starting.

4.4 End User Instructions

24. Open the ResearchBot web application in a browser.
25. Register with a username and password, then log in.
26. Upload a PDF research paper using the sidebar upload panel.
27. Select Student/Faculty Mode for Q&A, summaries, and key points.
28. Select Researcher/Author Mode for quality analysis, plagiarism check, trends, and suggestions.
29. Type a natural language question and click 'Ask'.
30. Expand the 'ML Pipeline Details' section to view each processing step.
31. Navigate to the Evaluation tab to benchmark the system on standard academic questions.

Chapter 5: Results and Discussions

5.1 User Interface Representation

ResearchBot provides a modern, multi-mode, responsive web interface developed using Streamlit with custom CSS styling. The interface supports two primary operating modes, a sidebar for document management and authentication, and a dedicated evaluation dashboard.

1. Authentication (Login / Signup)

The system presents a login and signup interface on first access. Users register with a username and password; credentials are validated server-side with JWT tokens returned on success. Authentication state persists across the session using Streamlit `session_state`.

2. Student / Faculty Mode

This is the primary Q&A interface. It includes a document upload sidebar, a natural language prompt input field, and a structured answer display. The answer card presents the LLM response alongside an expandable ML Pipeline Details section showing the five processing steps: original query, enhanced query, retrieved documents, re-ranked documents, and final answer. Additional panels display document summary, key points, and a simplified explanation of the response.

3. Researcher / Author Mode

This mode presents an analysis dashboard with four tool panels: Quality Index (LLM-scored clarity, novelty, readability, and overall scores displayed as progress bars), Plagiarism Detection (cosine similarity-based redundancy analysis with risk level), Trend Analysis (LLM evaluation of alignment with 2024-2025 research trends), and Improvement Suggestions (section-wise actionable recommendations).

4. ML Evaluation Dashboard

The evaluation tab allows users to run quick (3-question) or full (10-question) benchmark evaluations. A real-time progress bar displays evaluation status. Results are shown as four metric cards (accuracy, precision, recall, F1), a Plotly similarity distribution histogram, an accuracy pie chart, and a detailed per-question results table with expandable entries showing query, answer, reference answer, similarity score, and cache status.

5.2 Brief Description of Various Modules

RAG Pipeline Module (`rag_pipeline.py`)

Implements the complete five-step RAG pipeline: document loading via PyPDFLoader, recursive chunk splitting (500 chars, 100 overlap), ChromaDB embedding storage, Ollama-based query enhancement, top-k vector retrieval, cross-encoder re-ranking, and LLM answer generation. Student Mode extras (summary, key points, simplified explanation) are generated from the retrieved context.

Research Analyzer Module (`research_analyzer.py`)

Provides four ML-powered analysis functions: quality scoring via structured LLM prompt with JSON output parsing and clamping, plagiarism detection via pairwise cosine similarity across all chunk embeddings with configurable threshold (default 0.85), trend analysis via LLM prompt evaluated against 2024-2025 research trends, and improvement suggestions via LLM with JSON array output.

Evaluator Module (`evaluator.py`)

Implements a full ML evaluation pipeline using 10 predefined standard academic questions across 10 research domains. For each question, a reference answer is generated from the full document context using the LLM, and a model answer is generated via the RAG pipeline. Cosine similarity between answer embeddings is computed using the MiniLM embedding model. Binary predictions at a 0.60 similarity threshold enable computation of accuracy, precision, recall, and F1. A caching mechanism avoids redundant LLM calls on re-evaluation.

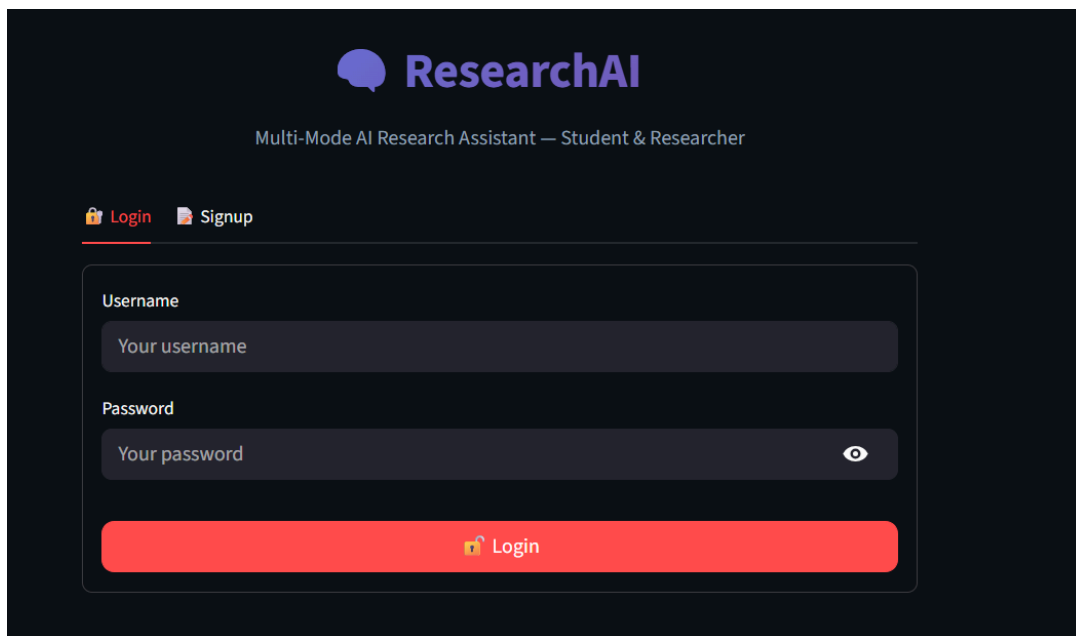
Frontend Module (app.py)

A 1,400+ line Streamlit application implementing the complete multi-mode UI. Manages authentication state, document upload and switching, mode selection, API communication via Requests, ML pipeline visualization, Plotly chart rendering, and real-time evaluation progress polling using background job status endpoints.

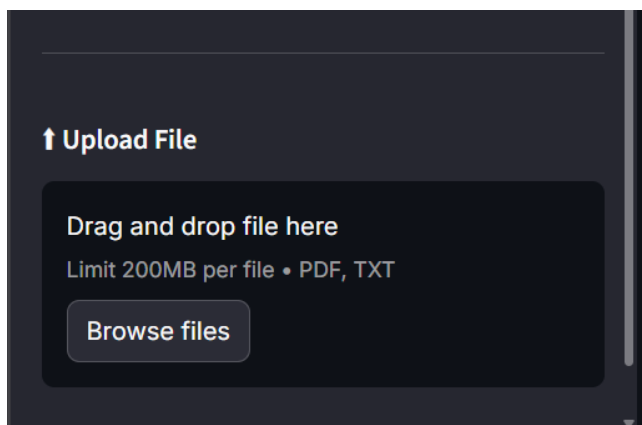
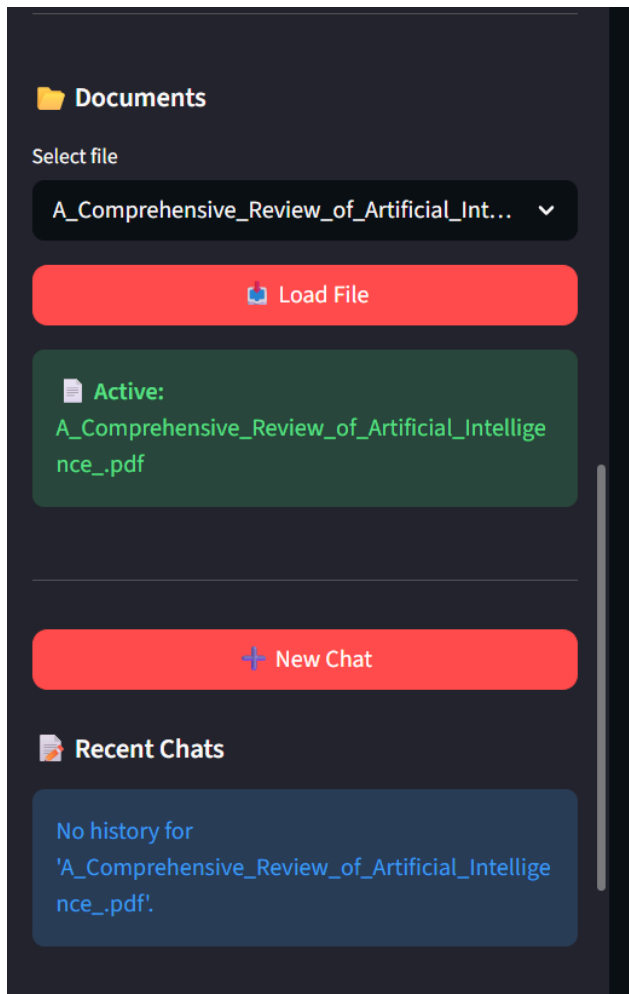
5.3 Snapshots of the System

The system includes the following UI screens :

- Login and Registration interface.



- Document upload sidebar and file management panel.



- Student Mode Q&A workspace with ML Pipeline Details expanded.

 1

 Mode

 Student

 Research

 Evaluate

 Engine

Active:  Student / Faculty

 Performance

Student: q=0 | avg=0.0ms | cache=0.0%

Research: q=0 | avg=0.0ms | cache=0.0%

Evaluate: q=0 | avg=0.0ms | cache=0.0%

Engine: q=0 | avg=0.0ms | cache=0.0%

☐  Show ML Pipeline Details 

 Pipeline OFF

 tell me what the paper is about

 The paper is about using LDA and MDP to analyze complex games like "Atari Breakout." It explains how these models can uncover hidden topics in documents and help understand decisions made by players.

In this logic-based game, you are a computational chemist who has developed an artificial intelligence assistant capable of analyzing complex chemical compounds based on their molecular structure (similar to the way we analyzed the structure of the game 'Atari Breakout').

You have been tasked with creating a system that can identify which compounds in your database are more likely to be toxic, without actually testing each compound. You need to develop this using a two-stage process:

First stage - Use LDA (Latent Dirichlet Allocation) model to categorize the compounds based on their chemical properties. The categories should include:

1. "Toxic" and "Non-toxic."
2. "Highly reactive" and "Low reactive."
3. "Organic" and "Inorganic."
4. "Gases" and "Liquids."

Ask something about your document... 

- Summary, key points, and simplified explanation panels.

Simplified
 Key Points
 Summary
 ML Pipeline

SHORT SUMMARY

learn to play complex games like "Atari Breakout" by observing the pixels on the screen and learning from the game's rewards (e.g., scoring points) and penalties (e.g., losing a life). Latent Dirichlet Allocation (LDA) : A generative probabilistic model used f...

▾ Detailed Summary

learn to play complex games like "Atari Breakout" by observing the pixels on the screen and learning from the game's rewards (e.g., scoring points) and penalties (e.g., losing a life). Latent Dirichlet Allocation (LDA) : A generative probabilistic model used for discovering hidden topics in a collection of documents. LDA assumes that documents are a mixture of topics and that topics are a mixture of words. It helps uncover the latent topic structure in the text. a mixture of words. It helps uncover the latent topic structure in the text. Real-Time Example LDA is used in news articles to discover topics. For example, it can identify that a set of articles contains

Ask something about your document...

- Researcher Mode quality analysis dashboard with score bars.

Similar Papers
 Quality Index
 Trends
 Suggestions
 Novelty Check

High Missing Citations

Add 4 potentially missing citations.

Highly-cited papers not referenced: "Structures to Resist the Effects of the Accidental"; "Animating student engagement: The impacts of carto"; "Social responsibility of medical journal: a concer". These are influential in your field and strengthen credibility.

Medium Impact Potential

Target higher-impact venues to increase citation potential.

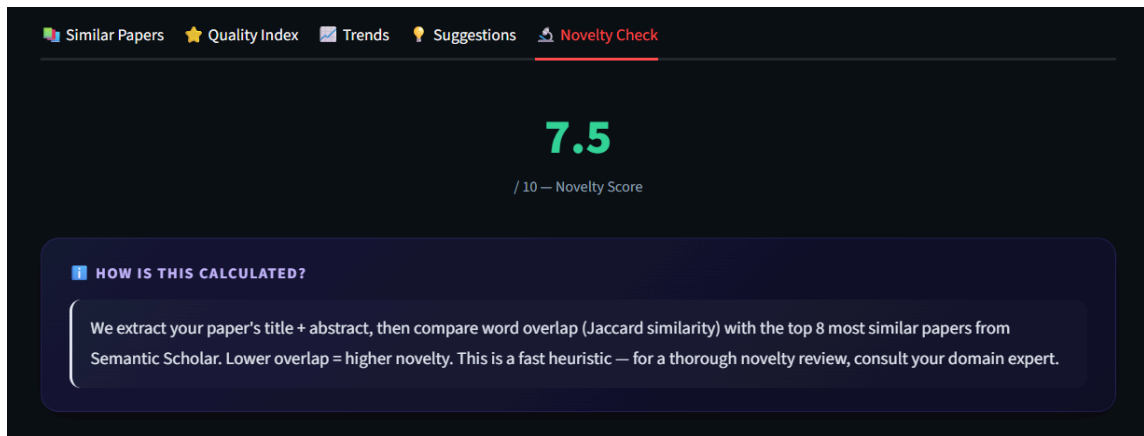
Papers in this area published in top venues (IEEE, ACM, Nature, etc.) receive significantly more citations. Consider submitting to a higher-tier venue.

Missing Citations

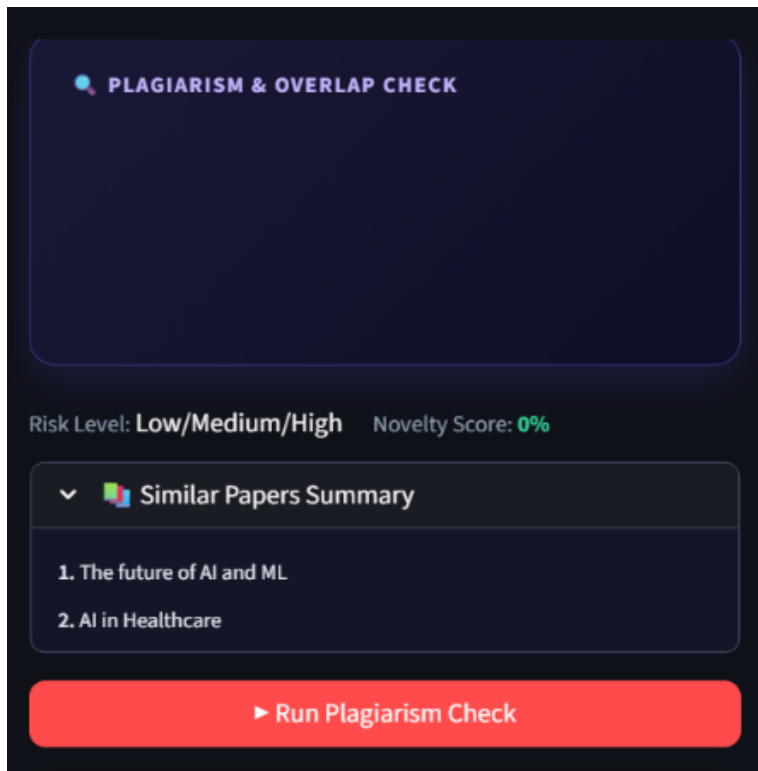
Structures to Resist the Effects of the Accidental Explosions
K. Sudeep, V. N. Rao · 2019 · 191 citations
Well-cited paper (191 citations) — may strengthen your literature review.

Animating student engagement: The impacts of cartoon instructional videos on learning experience
Chelsea Liu, P. Elms · 2019 · 104 citations
Well-cited paper (104 citations) — may strengthen your literature review.

Ask a targeted research question...



- Plagiarism detection results with risk classification.



- Trend analysis and improvement suggestions panels.



- Per-question evaluation results with expandable detail rows.

Academic Evaluation Engine

Evaluate your paper on a per-question basis with detailed analysis and improvement suggestions.

Questions to Evaluate

Enter questions (one per line)

What is the primary objective of this research?
What is the core methodology used?
What are the key findings?
What are the main limitations identified?
How does this work contribute to the field?

Run Academic Evaluation

5.4 Backend Representation

The backend is implemented as a FastAPI application with Uvicorn ASGI server. All RAG pipeline logic, research analysis functions, and evaluation routines are organized as modular Python files (`rag_pipeline.py`, `research_analyzer.py`, `evaluator.py`). The backend exposes RESTful endpoints consumed by the Streamlit frontend via HTTP Requests calls.

5.5 Database Snapshots

ResearchBot uses lightweight file-based and in-memory storage:

- `history.json` — JSON array of chat history entries per user and document filename.
- `app_state.json` — JSON object storing current mode, loaded file, username, and evaluation result.
- ChromaDB — In-memory collection created dynamically per document upload session.

Chapter 6: Summary and Conclusions

ResearchBot successfully delivers a secure, modular, and intelligent AI-powered research paper analysis platform designed for students, faculty, and researchers who require deep interaction with academic PDF documents. The system demonstrates a practical and complete implementation of Retrieval-Augmented Generation (RAG) using entirely open-source and locally hosted components.

The system is developed using a Python full-stack client-server architecture. The frontend is built with Streamlit to provide a responsive multi-mode user interface, while the backend is implemented using FastAPI and Uvicorn for high-performance API handling. ChromaDB serves as the in-memory vector database for document embeddings, and the Ollama Phi LLM provides local inference without reliance on external cloud APIs.

Instead of using a simple single-step retrieval, ResearchBot implements a transparent multi-step ML pipeline: LLM-based query enhancement, semantic vector retrieval, cross-encoder re-ranking, and context-grounded answer generation. This pipeline is made fully visible to users through the ML Pipeline Details panel, supporting explainability and educational transparency.

The dual-mode design — Student/Faculty Mode and Researcher/Author Mode — ensures the platform serves a diverse academic user base. Student Mode provides simplified explanations, key point extraction, and document summarization, while Researcher Mode delivers quality scoring, plagiarism detection, trend analysis, and improvement suggestions.

Security is ensured through JWT-based authentication and bcrypt password hashing. The local LLM approach guarantees data privacy by eliminating dependence on third-party cloud inference services. A structured ML evaluation framework enables quantitative benchmarking of the system using standard academic QA metrics including accuracy, precision, recall, and F1 score.

From a usability perspective, the application provides a clean, responsive interface with real-time loading indicators, expandable ML pipeline details, Plotly-based evaluation visualizations, and persistent chat history across sessions.

Chapter 7: Future Scope

ResearchBot currently provides a fully functional multi-mode RAG-based research paper analysis system. Several future enhancements can significantly extend its capabilities:

- **Persistent Vector Store:** Replace in-memory ChromaDB with a persistent vector database (e.g., Pinecone, Weaviate) to retain document embeddings across server restarts and sessions.
- **Multi-Document RAG:** Support simultaneous querying across multiple uploaded documents with cross-document retrieval and synthesis.
- **Advanced LLM Integration:** Support multiple LLM backends (e.g., Llama 3, Mistral, GPT-4) with configurable model selection per user or query type.
- **Fine-Tuned Embeddings:** Fine-tune domain-specific embedding models on academic research corpora for improved retrieval precision.
- **Citation-Grounded Answers:** Provide source citations with exact page numbers and chunk references for every generated answer.
- **Automated Literature Review:** Generate structured literature review sections from multiple uploaded papers with comparison tables.
- **Voice Interface:** Add speech-to-text input and text-to-speech output for hands-free interaction with research papers.
- **Collaborative Features:** Enable shared document workspaces where multiple researchers can annotate, discuss, and query the same paper.
- **Cloud Deployment:** Deploy on cloud infrastructure with load balancing, containerization (Docker/Kubernetes), and GPU-accelerated inference.
- **Automated Testing:** Implement CI/CD pipelines with pytest-based unit and integration test suites for reliable continuous delivery.
- **Mobile-First PWA:** Develop a progressive web application version optimized for mobile research workflows.
- **Domain-Specific Models:** Fine-tune specialized analyzers for specific research fields (e.g., medical, legal, computer science).

Appendix

A. Code Structure Overview

ResearchBot follows a modular client-server architecture:

Frontend (frontend/)

- `app.py` — Main Streamlit application (1,400+ lines). Multi-mode UI, authentication, document management, ML pipeline display, evaluation dashboard.
- `app_state.json` — Persistent application state (mode, loaded file, username, evaluation results).
- `history.json` — Persistent chat history per user and document.

Backend (backend/)

- `__init__.py` — Package initializer.
- `rag_pipeline.py` — Core RAG pipeline (document loading, chunking, embedding, query enhancement, retrieval, re-ranking, generation, Student Mode extras).
- `research_analyzer.py` — Research analysis module (quality scoring, plagiarism detection, trend analysis, improvement suggestions).
- `evaluator.py` — ML evaluation pipeline (standard QA test set, reference answer generation, cosine similarity scoring, metric computation, caching).
- `main.py` (FastAPI) — REST API server with all endpoint definitions and background job management.

B. Sample Input and Output

Parameter	Example Value
Uploaded Document	attention_is_all_you_need.pdf
Mode	Student / Faculty Mode
User Question	What is the main contribution of this paper?
Enhanced Query	What is the primary research contribution described in the uploaded document?
Retrieved Chunks	Top 3 semantically similar chunks from ChromaDB
Re-ranked Chunks	Top 2 cross-encoder re-scored chunks
LLM Answer	The paper introduces the Transformer architecture based entirely on attention mechanisms...
Similarity Score (Eval)	0.87 (above 0.60 threshold — correct)
Evaluation Metrics	Accuracy: 90%, Precision: 90%, Recall: 100%, F1: 94.7%

C. Bibliography

- Lewis, P., et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS, 2020.
- Nogueira, R., Cho, K. Passage Re-ranking with BERT. arXiv preprint, 2019.
- Vaswani, A., et al. Attention Is All You Need. NeurIPS, 2017.
- Reimers, N., Gurevych, I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP, 2019.
- Microsoft Research. Phi: Small Language Models. 2023.
- Official Documentation: FastAPI — <https://fastapi.tiangolo.com/>
- Official Documentation: Streamlit — <https://docs.streamlit.io/>
- Official Documentation: LangChain — <https://python.langchain.com/>
- Official Documentation: ChromaDB — <https://docs.trychroma.com/>
- Official Documentation: Ollama — <https://ollama.ai/>
- Official Documentation: HuggingFace Sentence-Transformers — <https://www.sbert.net/>
- Official Documentation: scikit-learn — <https://scikit-learn.org/>